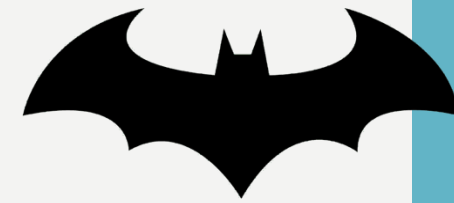




TP3 – LE SYSTÈME DE JETONS DE LA BATCAVE

PROTOCOLES D'AUTHENTIFICATION

CONTEXTE



Alfred et Batman font face à une crise majeure :

Lors de la dernière attaque du Joker, le serveur de la Batcave a totalement saturé sa mémoire vive sous une pluie de requêtes. Pire encore, Batman se plaint d'une expérience utilisateur (UX) catastrophique : lorsqu'il est en intervention dans la Batmobile, son accès se coupe brutalement à l'expiration de sa session, l'obligeant à retaper ses identifiants en plein combat.

Votre objectif consiste à migrer l'architecture de la Batcave vers un modèle *stateless* sécurisé et invisible pour l'utilisateur grâce au couple JWT *accessToken* / *refreshToken*.



INSTRUCTIONS

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

ÉTAPE 1 : MODÉLISATION DU FLUX

Afin de ne pas foncer tête baissée dans le code, vous devez concevoir l'architecture logique de votre système de renouvellement.

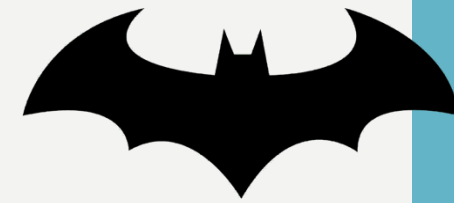
Réalisez un **schéma libre** (schéma logique, diagramme de séquence manuscrit pris en photo ou autre) illustrant le cycle de vie complet de l'authentification et du rafraîchissement.

Ce schéma doit faire apparaître les acteurs (*Navigateur/Frontend, Middleware, API/Serveur Express, Base de données SQLite*) et illustrer graphiquement les mécanismes suivants :

- Le `POST /login` qui génère et dépose les deux cookies distincts (`token` et `refreshToken`) ;
- Le rejet 401 par le middleware lorsque l'`accessToken` expire ;
- L'appel automatique en arrière-plan à la route `POST /refresh` ;
- Le rejeu de la requête initiale de manière transparente.

Ajoutez ce schéma, sous forme d'image, directement à la racine de votre dépôt Git.

ÉTAPE 2 : REFACTO



Faites évoluer votre fichier `config/db.js` pour y ajouter une table `refresh_tokens`.

Modifiez votre route `POST /login`:

- Supprimez toute référence aux sessions en RAM ;
- Générez un ***accessToken*** signé avec votre **clé secrète**. Pour vos tests en direct, donnez-lui une durée de vie très courte de **15 secondes** ;
- Générez un ***refreshToken*** opaque (chaîne aléatoire cryptographique) valide pour **7 jours**, et stockez-le dans votre table SQLite ;
- Envoyez les deux jetons au client avec deux cookies sécurisés distincts.

ÉTAPE 3 : LE RAFRAICHISSEMENT ET LA RÉVOCATION

Implémentez la route `POST /api/auth/refresh`. Elle doit intercepter le cookie `refreshToken`, vérifier s'il existe en base et s'il n'est pas expiré. Si tout est valide, elle réémet un cookie `accessToken` pour 15 secondes supplémentaires.

Batman a émis une consigne stricte : *"Si mon Bat-phone est volé par Harley Quinn, la déconnexion doit immédiatement m'assurer qu'elle n'a plus accès au système."*

- Modifiez votre route de déconnexion pour qu'elle exécute un `DELETE` en BDD de ce `refreshToken` spécifique avant d'effacer les cookies du client.

ÉTAPE 4 : RÉSILIENCE FRONTEND

Dans votre fichier `public/dashboard.js`, remplacez votre logique d'appel API classique par un mécanisme de rejeu ("Retry Pattern") :

1. Créez une fonction asynchrone qui encapsule le fetch natif ;
2. Si l'API répond un statut 401 (JWT expiré), la fonction doit automatiquement mettre la requête de l'utilisateur "en pause", appeler `POST /api/auth/refresh` en tâche de fond, puis rejouer la requête initiale si le rafraîchissement a réussi ;
3. Si le rafraîchissement échoue (*refreshToken* expiré ou supprimé de la BDD lors d'une déconnexion) redirigez vers `login.html`.

ÉTAPE 5 : MODIFIER LE MOT DE PASSE

Implémentez une route permettant à un utilisateur connecté de modifier son mot de passe :

- Créez `POST /api/auth/change-password` (accessible uniquement via le middleware d'authentification).
- **Double validation (Backend) :**
 1. **Authenticité** : Vérifiez avec `bcrypt.compare()` que l'ancien mot de passe fourni correspond au hash stocké en base.
 2. **Robustesse ANSSI** : Validez via une Regex stricte que le nouveau mot de passe contient au moins **12 caractères**, 1 majuscule, 1 minuscule, 1 chiffre et 1 caractère spécial.

💡 Hachez le nouveau mot de passe avec Bcrypt avant de l'enregistrer en base.

🚫 L'interdiction des mots de passe faibles se décide et se valide **exclusivement sur le serveur** jamais uniquement dans le frontend !

ÉTAPE 6 : VALIDATION

Pour valider ce TP, vous devez être capables de montrer les étapes suivantes dans votre onglet **Application / Cookies** de vos DevTools :

1. Vous vous connectez : les deux cookies apparaissent ;
2. Vous attendez 15 secondes sans toucher à rien : l'*accessToken* expire et son cookie s'efface. Le *refreshToken* reste présent ;
3. Vous effectuez une action sur le tableau de bord (rechargement ou clic d'action) : votre console affiche le rafraîchissement transparent, le cookie `token` réapparaît et vos données utilisateur restent affichées sans déconnexion ;
4. Vous cliquez sur déconnexion : les cookies s'effacent et la ligne est bien supprimée de votre table SQLite.



POUR ALLER PLUS LOIN

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

CHALLENGE 1 : DÉTECTION DE VOL ET ROTATION DES JETONS

Seulement pour les étudiants ayant terminé les phases précédentes !

Le plus grand danger d'un *refreshToken* est son vol. Si un espion du Pingouin dérobe un *refreshToken* valide, il peut générer des jetons d'accès indéfiniment.

Pour parer à cela, implémentez la **rotation de *refreshToken*** :

- À chaque fois que la route `/refresh` est appelée, le serveur doit **détruire** le *refreshToken* utilisé et en générer un **nouveau** qu'il renvoie au client.
- Si le serveur reçoit un *refreshToken* qui a déjà été utilisé (ce qui nécessite d'ajouter un statut en BDD), cela signifie qu'un attaquant ET la victime possèdent le même jeton. Le serveur doit supprimer *tous* les *refreshTokens* de cet utilisateur en BDD pour déconnecter immédiatement tous les appareils par sécurité.

CHALLENGE 2 : LE PROTOCOLE DE DOUBLE VALIDATION

Seulement pour les étudiants ayant terminé les phases précédentes !

Pour accéder aux commandes critiques de la Batmobile, l'identifiant et le mot de passe ne suffisent plus. Il faut une deuxième preuve d'identité :

- Modifiez le payload de votre premier JWT généré lors de la connexion pour y ajouter une propriété : `is2FAVerified: false`.
- Créez une route protégée `/api/user/secret-batmobile` qui vérifie ce paramètre. Si le middleware lit `is2FAVerified: false`, il bloque l'accès (même si le JWT est valide). L'utilisateur doit d'abord valider une étape intermédiaire (simuler la saisie d'un code secret reçu par Alfred) pour obtenir un JWT final contenant `is2FAVerified: true`.

CHALLENGE 3 : SIMULATION DE LA BAT-ARMURE

Seulement pour les étudiants ayant terminé les phases précédentes !

Batman conçoit une nouvelle armure connectée. Cette armure doit pouvoir interroger l'API de votre Batcave pour récupérer son profil (`/api/user/me`) mais elle n'a pas de système de gestion de cookies :

- Modifiez votre middleware `checkJWT()` pour qu'il accepte une deuxième méthode de transmission du jeton d'accès : l'en-tête `HTTP Authorization: Bearer <JWT_TOKEN>`.
- Utilisez un outil comme [Postman](#) ou [curl](#) pour requêter votre API protégée en y injectant manuellement le JWT dans les en-têtes.

Vous venez de poser la première pierre d'une architecture découplée de type **OAuth2 / OpenID Connect** où le client n'est plus un simple navigateur.

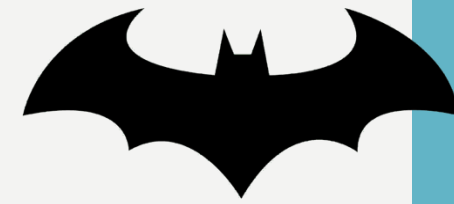


MODALITÉS D'ÉVALUATION ET CONTRÔLE CONTINU

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

« CODE CHECK »



Ce projet filé (TP1 + TP2) ne sera pas simplement noté sur votre dépôt Git.

👉 Lors de cette séance (et éventuellement la prochaine), j'échangerai avec **chaque étudiant en face-à-face** pendant 3 à 10 minutes sur le code et sa compréhension.

Les règles du jeu :

- Je pointerai une ligne de code que vous devrez m'expliquer précisément ;
- Attendez-vous à devoir justifier vos choix et être challengé sur votre maîtrise des concepts vus en cours.

❌ Si vous êtes incapable d'expliquer un bout de code présent sur dans votre dépôt ou si le vocabulaire technique utilisé à l'oral ne correspond pas à votre niveau supposé, la note en sera impactée.

A decorative wavy line on the left side of the slide, consisting of a light blue line and a white line.

DES QUESTIONS ?

CHEVALIER@CHRIS-FREELANCE.COM